

정적 분석

☉ Category	Security Analysis
≡ Description	Rule Based로 구현, java, js, flask python, ts 등 다양한 언어 지원
123 ID	Func-02
✧ Status	시작 전

Overview

- 목표 : 소스 코드의 구조를 분석하여 잠재적인 보안 취약점(CWE, OWASP Top 10 등)을 탐지함
- 대상 사용자 : 코드를 업로드하고 보안 점검을 받으려는 개발자 및 보안 담당자

User Flow & Logic

1. 분석할 방식을 선택 (직접 파일 업로드 or GitHub Repository URL 입력)
2. URL 입력 시 시스템은 해당 GitHub 저장소를 서버로 일시적으로 복제
3. 프로젝트의 폴더 구조를 스캔하여 분석 가능한 파일을 분류하고 전체 리스트를 생성
4. 설정된 보안 룰셋에 따라 모든 파일을 순회하며 취약한 패턴이나 위험한 함수 사용 탐지
5. 발견된 취약점을 [폴더 경로 > 파일명 > 줄 번호] 순으로 정리하여 프로젝트 전체의 보안 지도 생성
6. 분석된 기초 데이터를 요약하여 기록하고, 정밀 분석을 위해 AI 기반 분석 엔진으로 데이터 전달

Data Specification

Input

- Source Code → File (분석 대상)
- Project Name → String (사용자가 지정한 프젝명)
- Target_Language → Select (Python, JS, C 등)

Output

- Detection Results → Array (발견된 취약점 정보의 배열)
- Severity Counts → Object (위험도별 취약점 개수)
- Scan Duration → Float (분석에 소요 된 시간)

Exception Handling

- 지원하지 않는 언어 : 플랫폼에서 분석할 수 없는 언어 파일이 올라오면 "지원되지 않는 언어 형식입니다" 출력
- 파일 크기 초과 : 너무 큰 용량의 파일 업로드 시 서버 부하 방지를 위해 업로드 차단
- 분석 실패 : 코드 구문 오류로 인해 분석 중단 시 원인이 되는 코드 라인을 안내하고 분석 재시도 유도
- 잘못된 주소 : 잘못된 Github 주소거나 접근 권한이 없는 경우 "저장소에 접근할 수 없습니다" 메시지 출력

Security Requirements

- 임시 데이터 삭제 : 분석을 위해 서버에 임시로 가져온 파일들을 분석이 끝나고 결과 보고서 생성 후 즉시 영구 삭제
- 토큰 보호 : 사용자가 입력한 깃허브 접근 토큰은 암호화 하여 사용, 서버 데이터 베이스에 따로 저장하지 않음